

# Notebook/Script to OGC/DPS Application Package Generator

## Executive Summary

This project prototypes an automated workflow that takes a scientist's notebook, Python script, or executable entrypoint and converts it into a first-draft MAAP DPS / OGC Application Package. The generator performs static analysis, infers runtime inputs and dependencies, creates packaging artifacts, and now includes AI-assisted optimized access planning supported by MCP-ready dataset fact tools.

The main goal is to reduce the manual work required to move science code from an interactive development environment into a reproducible, scalable, cloud-executable package.

## Problem Statement

Scientists often start with working analysis code, but deploying that code as a DPS or OGC application requires many additional files and decisions:

- ? runtime wrapper scripts
- ? build scripts
- ? dependency/environment files
- ? Dockerfile
- ? algorithm metadata
- ? CWL command and workflow descriptors
- ? input/output schemas
- ? data-access credentials and patterns
- ? validation and reporting artifacts

Creating these manually is slow, inconsistent, and error-prone. It also requires knowledge of data access patterns, cloud-hosted NASA data, MAAP runtime behavior, and OGC packaging conventions.

## Project Goal

The project goal is:

> Given user science code, automatically infer enough information to generate a runnable DPS/OGC package, explain risks, and recommend efficient data-access strategies.

The generator is designed to work with:

- ? Python scripts
- ? notebooks converted to Python
- ? executable .run / shell entrypoints
- ? NISAR-style HDF5 workflows
- ? OPERA-style NetCDF/COG workflows

## Repository Structure

The repository now keeps source files and templates only. Generated package directories are intentionally ignored by git because they can be recreated at any time.

Key source folders:

generator/  
generate\_package.py  
access\_evidence.py  
access\_planner.py  
access\_plan\_validator.py  
access\_runtime.py  
access\_strategies.py  
dependency\_map.yml  
  
mcp\_server/  
access\_mcp\_server.py  
tools/  
cmr.py  
asset\_inspection.py  
access\_options.py  
cost\_estimator.py  
recommendation.py  
  
input/  
nisar\_access\_subset.py  
opera\_access\_structure.py  
  
templates/  
Dockerfile.template  
run.sh.template  
env.yml.template  
algorithm.yml.template  
commandline.cwl.template  
workflow.cwl.template  
  
tests/

Generated folders such as generated\_package/ and generated\_opera\_package/ are outputs, not source.

## End-to-End Workflow

The generator workflow is:

User script/notebook  
|  
v  
Static analysis  
|  
v  
Dependency and input inference  
|  
v  
Dataset fact extraction  
|  
v  
AI or fallback access planning  
|

v  
Validated strategy selection  
|  
v  
Template rendering  
|  
v  
Generated DPS/OGC package

## Static Analysis

The generator uses Python AST parsing and source inspection to identify:

- ? imports
- ? command-line arguments
- ? notebook/Papermill-style parameters
- ? data-access calls
- ? S3, HTTPS, CMR, STAC, and local file references
- ? output calls such as `to_zarr()` and `to_raster()`
- ? risky patterns such as interactive input, notebook magics, hardcoded local paths, and missing runtime declarations

This analysis is performed without running the user's science workflow.

## Generated Package Artifacts

For each input script, the generator can create:

app.yaml  
run.sh  
build.sh  
env.yml  
requirements.txt  
Dockerfile  
algorithm\_config.yaml  
algorithm.yml  
application.cwl  
workflow.cwl  
analysis.json  
dataset\_facts.json  
access\_plan.json  
access\_runtime.py  
llm\_analysis\_prompt.json  
report.md  
stac-input.json  
stac-output.json  
validate\_package.sh  
register\_dps.py  
publish\_ogc.py

These files are created from templates and inferred metadata.

## NISAR Demonstration

The NISAR example uses:

- ? NISAR GCOV HDF5 data
- ? Earthdata / MAAP / S3 credential access
- ? bbox subsetting
- ? Zarr output
- ? a manifest file describing the output

The generator adds safeguards so that a full granule is not accidentally written to a small notebook disk. A bbox or explicit full-granule flag is required.

Example generation:

```
python3 generator/generate_package.py
```

Example run:

```
cd generated_package
./run.sh \
--bbox "148325,5392805,158325,5402805" \
--bbox_crs "EPSG:32633"
```

## OPERA Demonstration

The OPERA example uses:

- ? OPERA DISP-S1 NetCDF
- ? direct S3 URL defaults
- ? MAAP and Earthaccess credential fallback
- ? xarray.open\_dataset(..., chunks="auto")
- ? water mask extraction
- ? index-window subsetting
- ? COG output

The OPERA script is self-describing through metadata constants such as:

```
APP_NAME = "opera_water_mask_to_cog"
APP_TARGET = "both"
APP_BASE_CONTAINER = "mas.maap-project.org/root/maap-workspaces/base_images/pangeo:v4.1.1"
```

Example generation:

```
python3 generator/generate_package.py input/opera_access_structure.py \
--output-dir generated_opera_package
```

Example run:

```
cd generated_opera_package
python opera_water_mask_to_cog.py \
```

```
--dest output \
--idx-window "0:1024,0:1024"
```

## AI Access Planning: Phase 1

Phase 1 introduced an optimized access planner.

The generator builds structured evidence from the script:

- ? imports
- ? URLs
- ? collections
- ? file formats
- ? operations
- ? input schema
- ? output schema
- ? resource hints
- ? static-analysis findings

It then selects a supported access strategy.

Supported strategies include:

- ? direct\_s3\_xarray
- ? direct\_s3\_h5py
- ? direct\_s3\_rasterio
- ? rasterio\_windowed\_read
- ? zarr\_open\_zarr
- ? virtual\_zarr\_access
- ? harmony\_subset
- ? https\_streaming
- ? https\_download\_fallback
- ? cmr\_search\_then\_s3

If OpenAI is enabled, the AI selects the strategy. If AI is unavailable or returns invalid output, the generator falls back to deterministic rules.

Output:

access\_plan.json

## MCP-Ready Dataset Facts: Phase 2

Phase 2 added MCP-ready dataset fact tools.

These tools are implemented as local Python functions but are structured so they can be exposed later through a true MCP server.

Tools include:

- ? get\_cmr\_collection
- ? get\_cmr\_granule
- ? inspect\_asset
- ? check\_access\_options

- ? estimate\_subset\_cost
- ? recommend\_access\_pattern

The generator writes:

dataset\_facts.json

This file records:

- ? CMR collection facts
- ? CMR granule facts
- ? S3 and HTTPS URLs
- ? inferred asset format
- ? variables and dimensions when inferable
- ? available access options
- ? subset-cost risk
- ? recommended access patterns

The AI planner then receives:

static evidence + dataset facts + allowed strategies

This makes the AI decision evidence-based instead of purely prompt-based.

## Strategy-Specific Runtime Helpers: Phase 3

Phase 3 added generation of:

access\_runtime.py

This file contains reusable helper functions selected from the access plan.

Examples:

- ? direct\_s3\_xarray generates open\_xarray\_dataset\_from\_s3(...)
- ? direct\_s3\_h5py generates download\_s3\_asset\_to\_tempfile(...)
- ? rasterio\_windowed\_read generates read\_raster\_window(...)
- ? zarr\_open\_zarr generates open\_zarr\_dataset(...)
- ? harmony\_subset generates Harmony request helper code
- ? https\_download\_fallback generates HTTPS download helper code

The current Phase 3 implementation is intentionally safe: it generates helper code but does not rewrite the scientist's original workflow automatically.

## OpenAI Usage

OpenAI can be used in two places:

1. **\*\*AI access planning\*\***  
- chooses the optimized access strategy from evidence and dataset facts
2. **\*\*LLM package review\*\***  
- reviews static analysis, package metadata, and access plan

Example:

```
python3 generator/generate_package.py input/opera_access_structure.py \
--output-dir generated_opera_package \
--ai-access-planner \
--access-planner-provider openai \
--llm-analysis \
--llm-provider openai
```

The AI is not allowed to generate arbitrary unsafe code. The selected strategy is validated against an allowlist before it is accepted.

## Validation and Testing

The project includes unit tests for:

- ? notebook parameter inference
- ? static-analysis findings
- ? manifest generation
- ? OPERA-style dashed CLI arguments
- ? executable .run handling
- ? access evidence extraction
- ? access-plan validation
- ? rule-based fallback planning
- ? MCP-ready dataset fact tools
- ? generated access\_runtime.py helpers

Recent validation included:

```
python3 -m unittest discover -s tests
python3 generator/generate_package.py
python3 generator/generate_package.py input/opera_access_structure.py --output-dir
generated_opera_package
python3 -m py_compile generator/*.py mcp_server/*.py mcp_server/tools/*.py input/*.py
generated_package/*.py generated_opera_package/*.py
generated_package/validate_package.sh
generated_opera_package/validate_package.sh
```

Manual NISAR smoke test successfully wrote:

```
output/nisar_subset.zarr
output/manifest.json
```

## Current Status

Implemented:

- ? static analysis
- ? dependency inference
- ? generated app manifest
- ? DPS/OGC template rendering

- ? OpenAI review
- ? AI access planner
- ? dataset facts tools
- ? MCP-ready tool dispatcher
- ? generated access plan
- ? generated runtime helper module
- ? NISAR and OPERA examples
- ? test coverage

Not yet implemented:

- ? full MCP protocol server integration
- ? automatic rewriting of science code to call `access_runtime.py`
- ? live file-structure inspection for every remote asset type
- ? Harmony service execution
- ? production registry publish flow

## Recommended Next Steps

1. Expose the MCP-ready tools through a true MCP server runtime.
2. Add richer live asset inspection for NetCDF, HDF5, Zarr, and GeoTIFF.
3. Add Harmony capability discovery and request generation.
4. Add optional code-rewrite mode to integrate `access_runtime.py` into generated scripts.
5. Add CI tests with mocked CMR metadata fixtures.
6. Demonstrate the full workflow on NISAR, OPERA, and a user-provided `.run` algorithm.

## Conclusion

This project demonstrates a practical path from user science code to generated OGC/DPS application packages. It combines deterministic static analysis, reusable templates, AI-assisted access planning, MCP-ready metadata tools, and validated package generation.

The most important design principle is that AI helps choose and explain the strategy, but deterministic validation and templates keep the generated package safe and reproducible.